Linear Regression

EECS 492

Introduction to Artificial Intelligence

Reminders

- This week's discussion on linear regression and non-parametric ML
- HW1 deadline is Feb 4

Today's Class

- Univariate Linear Regression
- Multivariate Linear Regression
- Regularization

Linear Regression

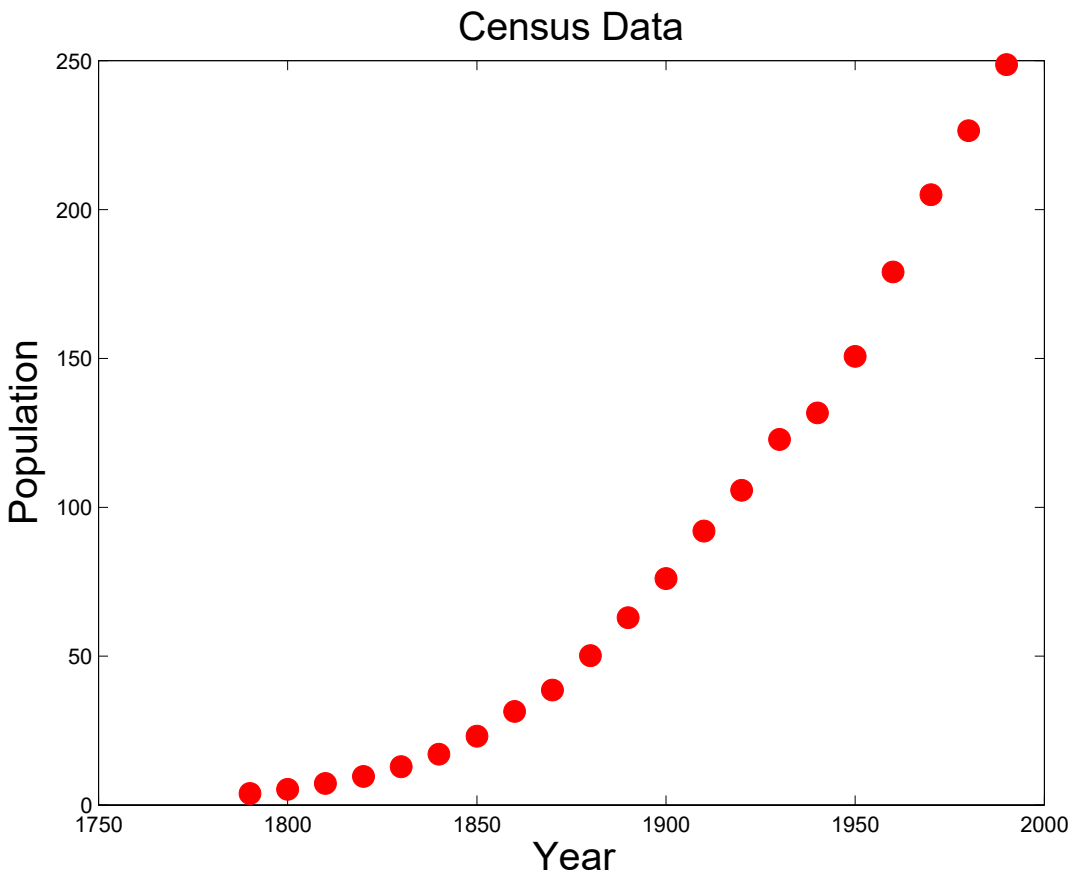
Classification and Regression

- We want to learn functions of the form:
  - $y = f(x)$

- The types of values that y takes affects the modeling that we use
  - Discrete y – classification
  - **Continuous y – regression**

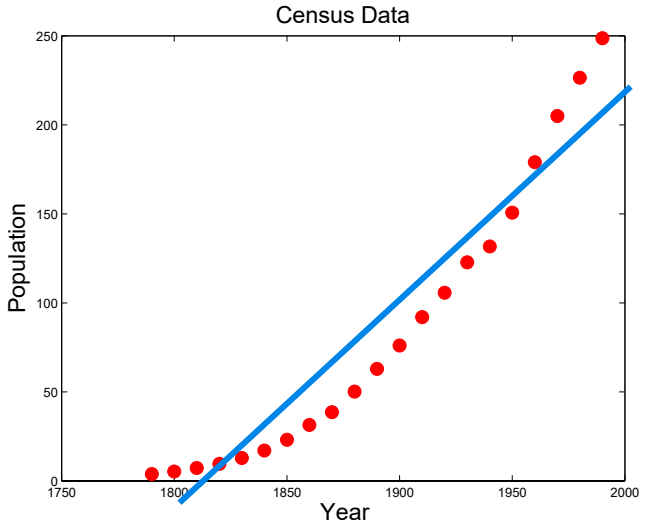
- The values of x can be continuous or discrete

Modeling Population Changes



How do we think about this goal?

- We may think we want a model that describes a fixed relationship
  - But, this won't allow for errors! Which is unrealistic in real-world data
- Alternatively, we would like to find a model that assumes natural variation in data
  - The model should capture the overall pattern and allow for **random fluctuations**
- Simple linear regression!
  - A straight-line relationship **plus some noise**

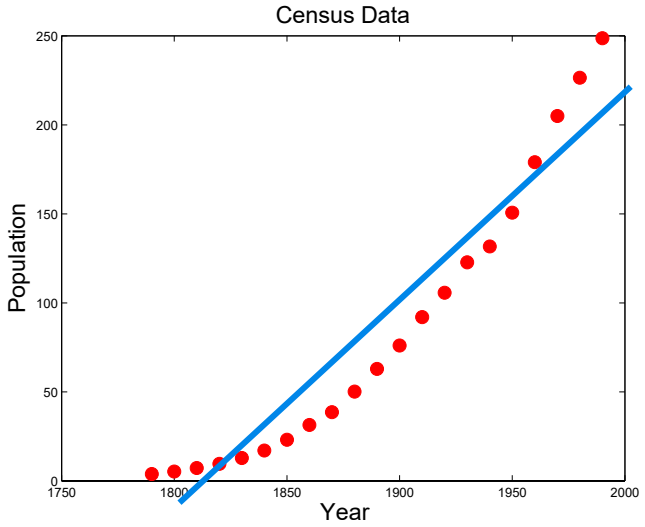


Linear Regression Assumptions

- Mean value of y for a given value of x graphs as a straight line

- Points deviate about this line of means by a random (positive or negative) amount equal to ϵ

- Formulation: $y = \underbrace{w_0 + w_1 x}_{\text{Mean value of y for a given x}} + \epsilon$ ← Random error



An aside: what are we assuming?

- Assumption 1:
  - Mean of probability distribution of ϵ is 0
  - Another way: average of errors over infinitely long series of experiments is 0

- Assumption 2:
  - Variance of probability distribution of ϵ is constant for all settings of independent variable x
  - For straight-line model: variance of ϵ is equal to a constant, σ^2, for all values of x

- Assumption 3:
  - Probability distribution of ϵ is normal (Gaussian)

- Assumption 4:
  - Errors associated with any two different observations are independent
  - Error of one value of y has no effect on other y's

An aside: what are we assuming?

- Assumption 1:
  - Mean of probability distribution of ϵ is 0
  - Another way: average of errors over infinit ts is 0
- Assumption 2:
  - Variance of n endent variable x
  - For stra ues of x
- Assumptio
  - Probabilit
- Assumption 4
  - Errors associa th any two different observations are independent
  - Error of one value of y has no effect on other y's

In practice – assumptions won't hold exactly!

Usually, this still results in reliable models.

Univariate Linear Regression

Notation:
- x: input feature
- y: output value
- w_o, w_1: weights
- h_w: prediction function

- Let's look at linear regression on one variable (univariate)
  - We previously defined: $y = f(x)$
  - Let $f(x)$ be a linear function of the form: $y = Ax + b$
  - We will think of the coefficients as weights. We can rewrite this with weights w_1 and w_0:

$$h_w(x) = w_1 x + w_0$$

- Our goal is to find the h_w that best fits these data
  - We will find the values of the weights $[w_0, w_1]$
  - Training goal: minimize the empirical loss

$$w^* = argmin_w Loss(h_w)$$

Empirical Loss

- L_2: squared-error loss function
- y_j: true output value for data point j
- x_j : input feature for data point j
- $h_w(x_j)$: predicted value for data point j

- It is traditional to use the squared-error loss function L_2

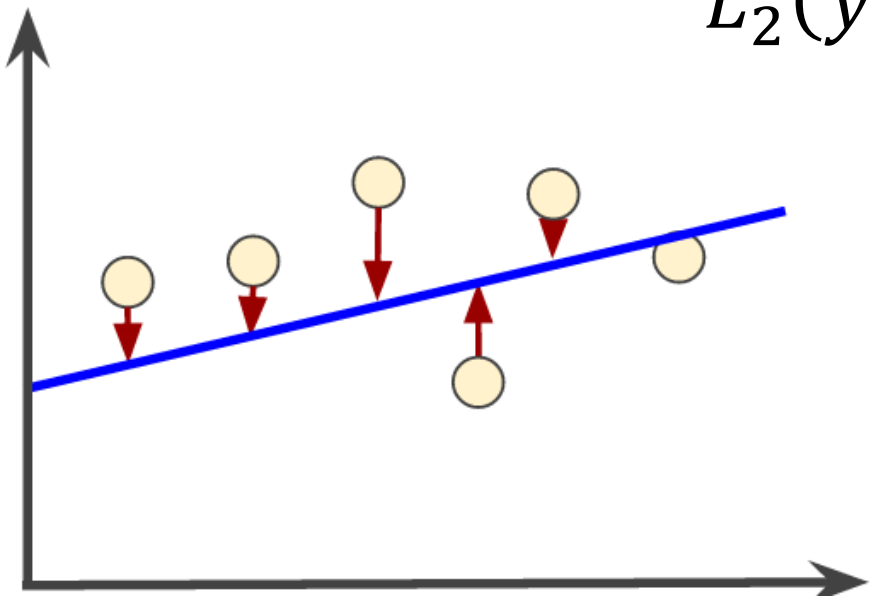
$$L_2(a, b) = (a - b)^2$$

- For one data point, we want the difference between our predicted value and the ground truth value:

$$L_2(y_i, h_w(x_i)) = (y_i - h_w(x_i))^2$$

The predicted value

The ground truth value



Empirical Loss for Univariate Linear Regression

- We want to sum up the L_2 loss over all n training examples:

$$Loss(h_w) = \sum_{i=1}^{n} L_2(y_i, h_w(x_i))$$

The predicted value

The ground truth value

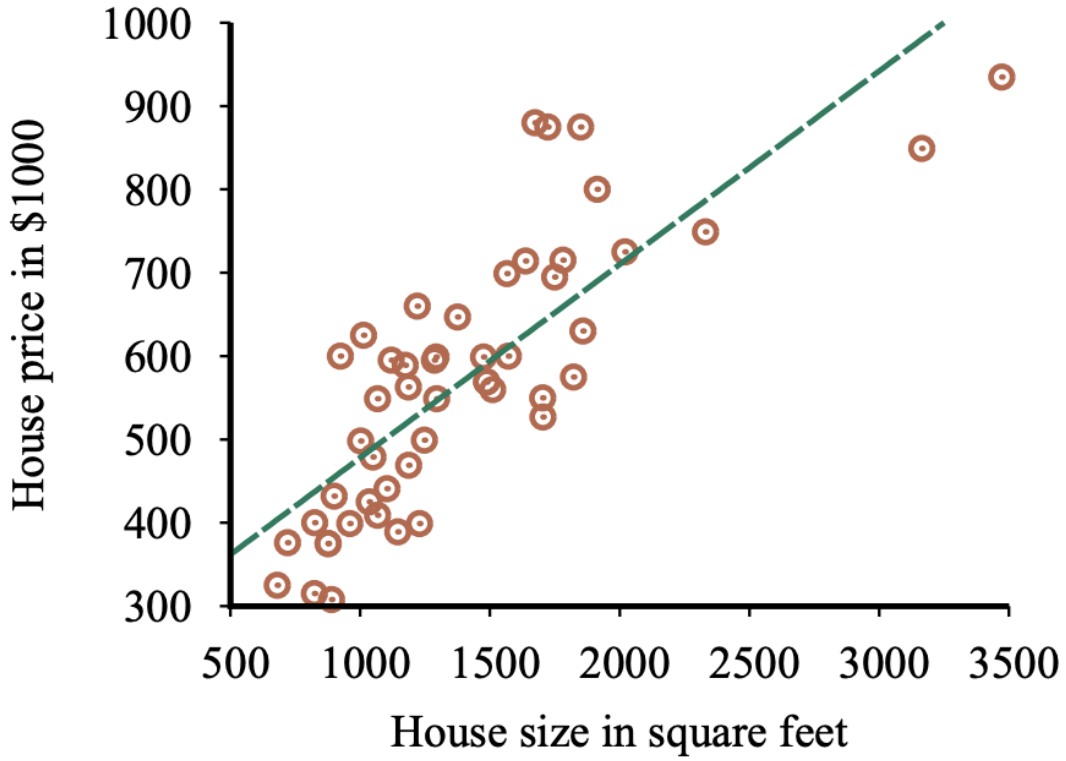
$$= \sum_{i=1}^{n} (y_i - h_w(x_i))^2$$

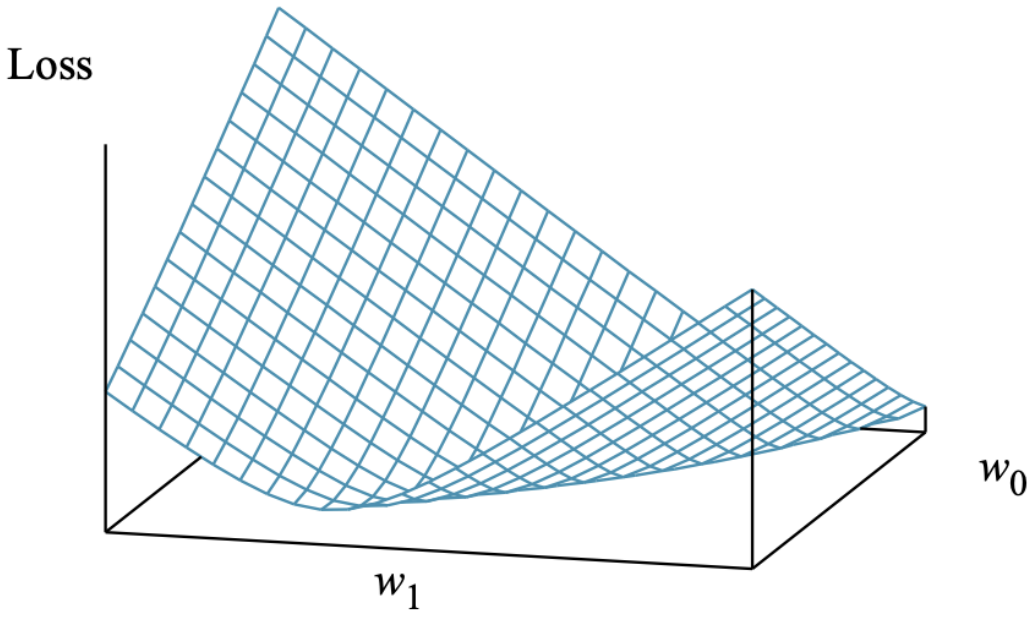
$$= \sum_{i=1}^{n} (y_i - (w_1 x_i + w_0))^2$$

- L_2: squared-error loss function
- n: number of training examples
- h_w: prediction function
- y_i: true output value for data point j
- x_i : input feature for data point j
- $h_w(x_i)$: predicted value for data point j
- w_o, w_1: weights

L2 loss also known as sum of squared errors

L2 loss guarantees convexity, i.e., no local minima





Note – the loss function is convex!

How do we solve this?

- Sum of squared errors: $Loss(h_w) = \sum_{i=1}^{n}[y_i - \underbrace{(w_0 + w_1 x_i)}]^2$

Our best guess, our estimate

Known value of our target

Consider all data points

How do we solve this?

- Sum of squared errors: $Loss(h_w) = \sum_{i=1}^{n} [y_i - (w_0 + w_1 x_i)]^2$

- Compute partial derivatives:
  - $\frac{\delta Loss}{\delta w_0} = \sum_{i=1}^{n} 2[y_i - (w_0 + w_1 x_i)](-1)$
  - $\frac{\delta Loss}{\delta w_1} = \sum_{i=1}^{n} 2[y_i - (w_0 + w_1 x_i)](-x_i)$

- Set partial derivatives equal to 0 and simplify …

How do we solve this?

- Sum of squared errors: $Loss(h_w) = \sum_{i=1}^{n}[y_i - (w_0 + w_1 x_i)]^2$

- Compute partial derivatives:
  - $\frac{\delta Loss}{\delta w_0} = \sum_{i=1}^{n} 2[y_i - (w_0 + w_1 x_i)](-1)$
  - $\frac{\delta Loss}{\delta w_1} = \sum_{i=1}^{n} 2[y_i - (w_0 + w_1 x_i)](-x_i)$

- Set partial derivatives equal to 0 and simplify …

$$w_1 = \frac{n(\sum x_i y_i) - (\sum x_i)(\sum y_i)}{n(\sum x_i^2) - (\sum x_i)^2}$$

$$w_0 = \frac{\sum y_i - w_1 \sum x_i}{n}$$

This is called the **analytical solution** for univariate linear regression

Multivariate Linear Regression

- h_w: prediction function
- $\boldsymbol{x_i}$: vector of input features for data point i
- $\boldsymbol{w}_0$: intercept weight
- $\boldsymbol{w}_j$: weight for feature j
- k: number of features
- $\boldsymbol{x}_{i,j}$: input feature j for data point i
- $\boldsymbol{w}$: vector of weights

- What about more than one feature?
  - Let's say we have k features:

$$h_w(x_i) = w_0 + \sum_{j=1}^{k} w_j x_{i,j}$$

- We can rewrite this to make it simpler!
  - Treat the intercept, w_0, as interacting with a dummy variable ($x_{i,0}$) which always equals 1:

$$h_w(x_i) = \sum_{j=0}^{k} w_j x_{i,j}$$

  - Simplify using vector notation:

$$h_w(x_i) = \sum_{j=0}^{k} w_j x_{i,j} = \boldsymbol{w}^T \boldsymbol{x}_i$$

Finding Optimal Weights

- h_w: prediction function
- $\boldsymbol{w}^*$: optimal weights
- $\boldsymbol{X}$: matrix of input features
- $\boldsymbol{y}$: vector of output values

- Again, we want weights that minimize empirical loss:

$$\boldsymbol{w}^* = argmin_{\boldsymbol{w}} Loss(h_w)$$

- Option #1: Solve it analytically

$$\boldsymbol{w}^* = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

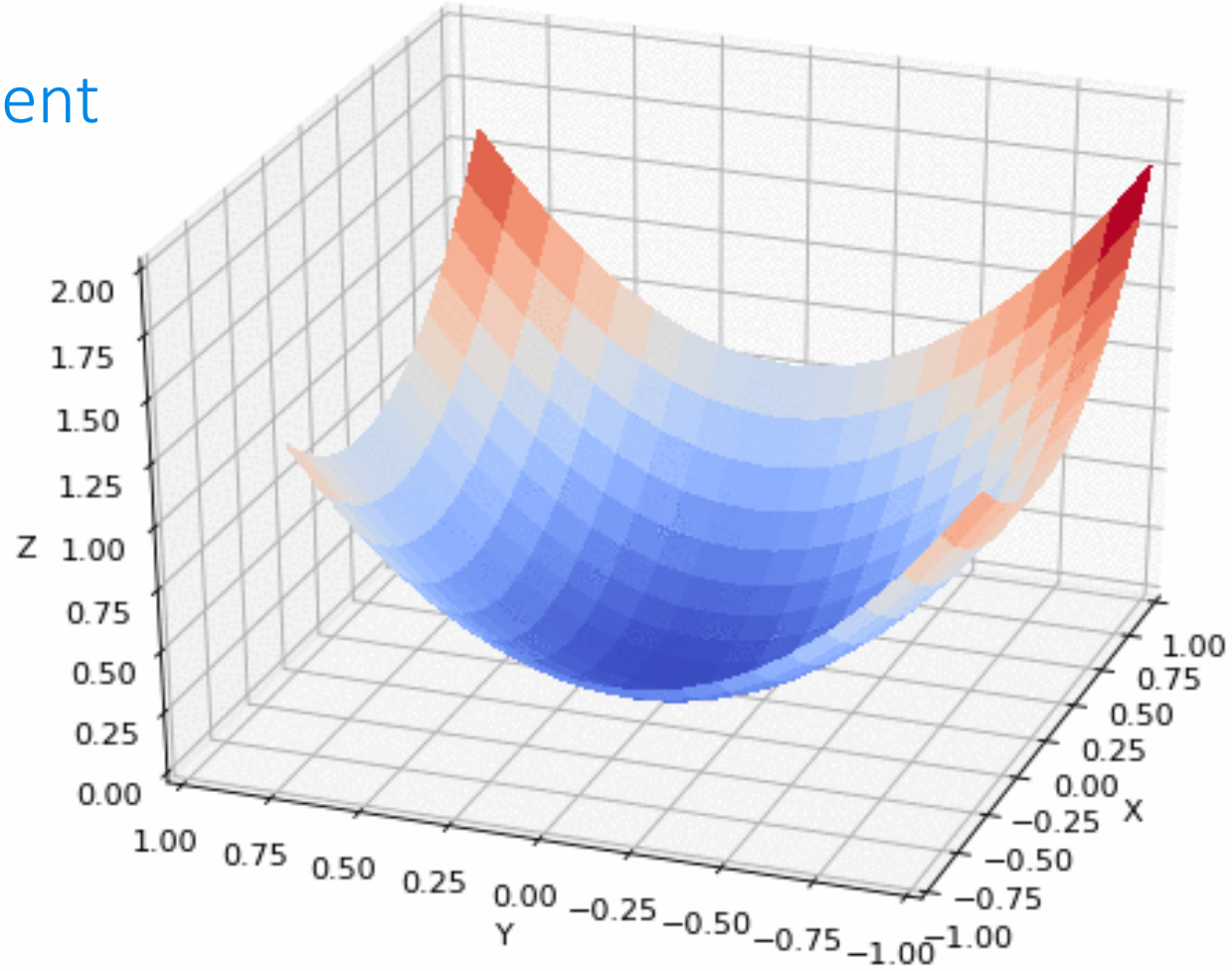
- $\boldsymbol{X}$ is a matrix of all the input features (each row is all the features for a single data point)
- $\boldsymbol{y}$ is a vector of true output values for each data point

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,k} \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_{n,1} & \dots & x_{n,k} \end{bmatrix}$$

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Finding Optimal Weights

- Option #2: Gradient descent



Animation from https://github.com/pvigier/gradient-descent

- $\boldsymbol{w}$: vector of weights
- $\boldsymbol{w}_j$: weight for feature j
- α: learning rate

Gradient Descent Algorithm

- Pick a random starting point $\mathbf{w}$ (a random vector of weights)
- For each weight, estimate the gradient $\frac{\partial}{\partial w_j} Loss(\boldsymbol{w})$
  - The gradient tells you which direction is downhill (for a two-dimensional line, this is the slope)
- For each weight, move a small amount in the steepest downhill direction
  - The learning rate α controls how far you move
  - $w_j \leftarrow w_j - \alpha \frac{\partial}{\partial w_j} Loss(\boldsymbol{w})$
- Repeat until convergence at a local minimum

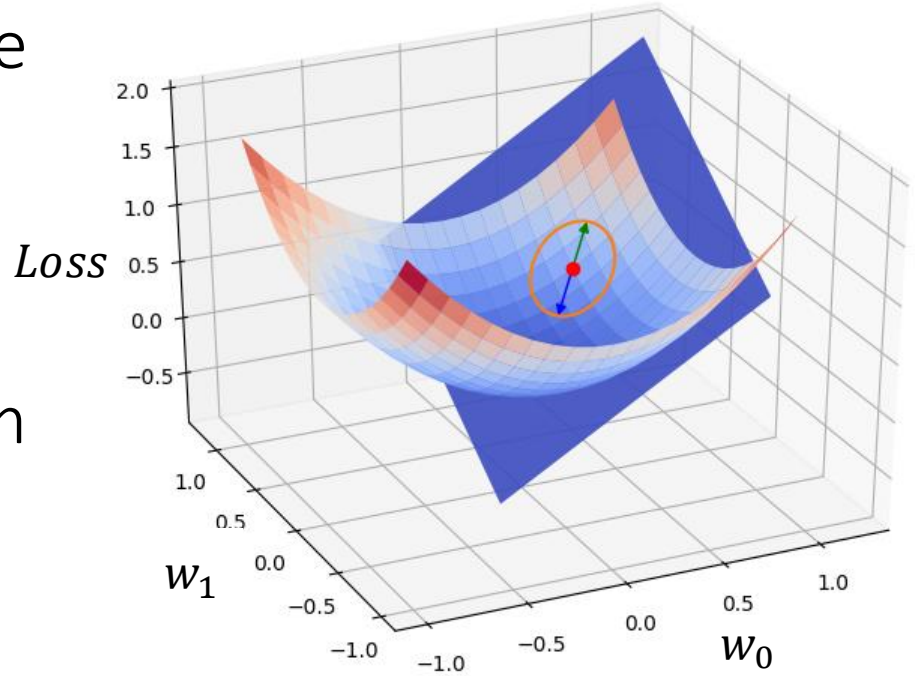


Image from https://github.com/pvigier/gradient-descent

Two Types of Gradient Descent

- Type 1: Batch gradient descent
  - Process all data points at once
  - Convergence is guaranteed given a small enough learning rate
  - Epoch – one step that covers all the training examples

- Updating bias weight:
  - $w_0 \leftarrow w_0 + \alpha \frac{1}{n} \sum_i \left(y_i - h_w(x_i)\right)$
- Updating other weights:
  - $w_j \leftarrow w_j + \alpha \frac{1}{n} \sum_i \left(y_i - h_w(x_i)\right) \times x_i$

- $\boldsymbol{w_j}$: weight for feature j
- α: learning rate
- y_i: true output value for data point i
- $h_w(x_i)$: predicted value for data point i
- $\boldsymbol{x_i}$: vector of input features for data point i
- $\boldsymbol{n}$: number of training examples

Two Types of Gradient Descent

- Type 2: Stochastic gradient descent (SGD)
  - Consider a single training point (or a small minibatch of training points) at a time
  - Take a step after each update
  - Convergence is not guaranteed

- Updating bias weight for a single training point x_i:
  - $w_0 \leftarrow w_0 + \alpha\big(y_i - h_w(x_i)\big)$
- Updating other weights for a single training point x_i:
  - $w_j \leftarrow w_j + \alpha\big(y_i - h_w(x_i)\big) \times x_i$

- $\boldsymbol{w_j}$: weight for feature j
- α: learning rate
- y_i: true output value for data point i
- $h_w(x_i)$: predicted value for data point i
- $\boldsymbol{x_i}$: vector of input features for data point i

When poll is active respond at **PollEv.com/492**

Thinking it through

- We want weights that minimize empirical loss:

$$\boldsymbol{w}^* = argmin_{\boldsymbol{w}} Loss(h_w)$$

- When should we use the analytical solution **vs** gradient descent?

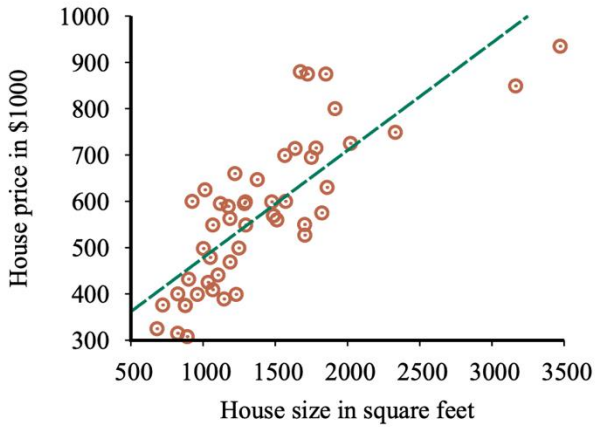
- When should we use batch gradient descent **vs** stochastic gradient descent?

Evaluating Regression Models

- R^2 statistic (coefficient of determination) - measures how well a regression model fits the data
  - Value of 1 – perfect relationship between true output and predicted output
  - Value of 0 – no relationship between true output and predicted output

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \overline{y})^2}$$

SS_{RES} – how much variance is due to the residual (error) in prediction
SS_{TOT} – total sum of squares; proportional to the variance of the data



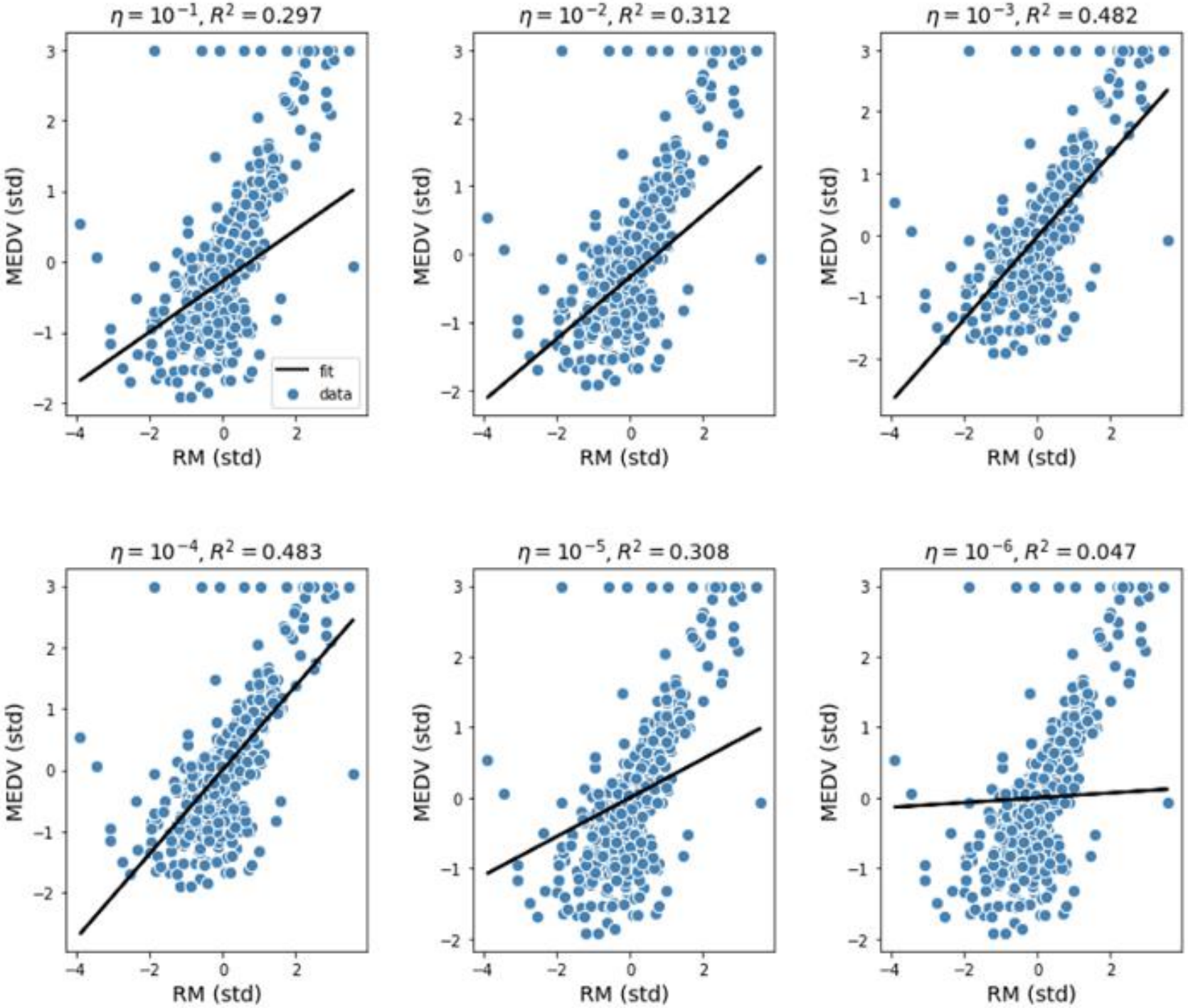


Image from https://benjaminobi.medium.com/what-really-is-r2-score-in-linear-regression-20cafdf5b87c

Overfitting and Regularization

- h_w: prediction function
- λ: regularization weight
- L_q: regularization function
- $\boldsymbol{w}$: vector of weights
- $\boldsymbol{w}_i$: weight for feature i
- q: regularization coefficient

- There is a real danger of overfitting
  - Especially true for multivariate regression!

Note that this L_1 and L_2 are different from the L_2 (squared loss) we saw earlier

- Regularization
  - Penalizes complex models

$$Cost(h) = EmpLoss(h) + \lambda Complexity(h)$$

- For linear functions: specify this as a function of the weights

$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

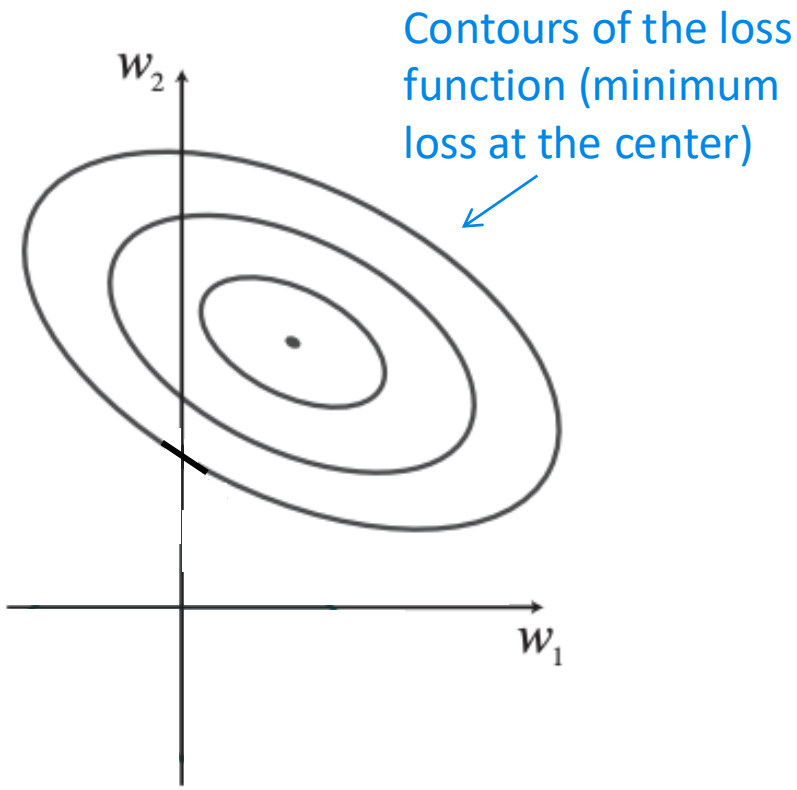
L_1 regularization, q = 1
L_2 regularization, q = 2
…

$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

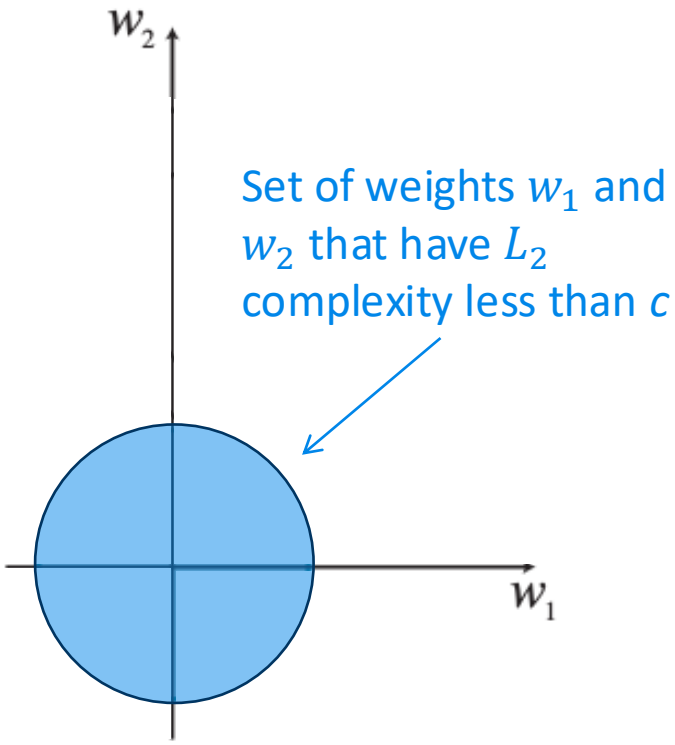
L_1 regularization, q = 1
L_2 regularization, q = 2
…

Visualizing L2 Regularization

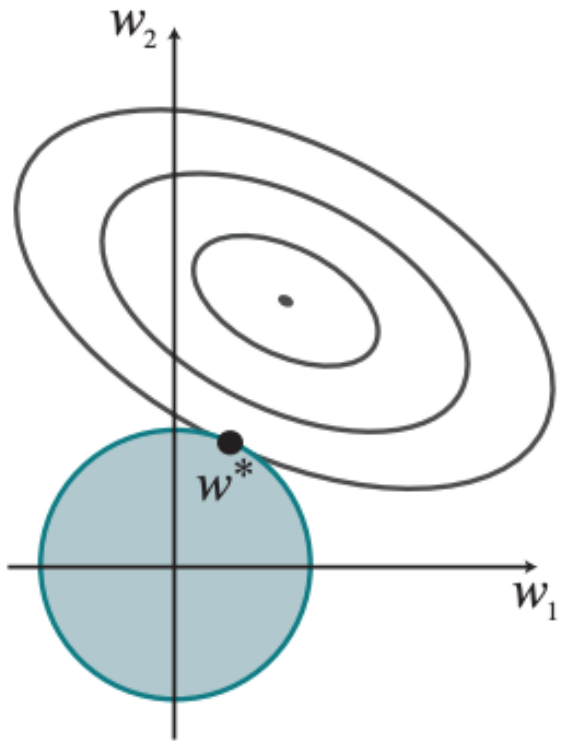
Let's visualize an example loss function with two weights:



L2 regularization will prioritize smaller weights:



Optimal weights with complexity less than c:



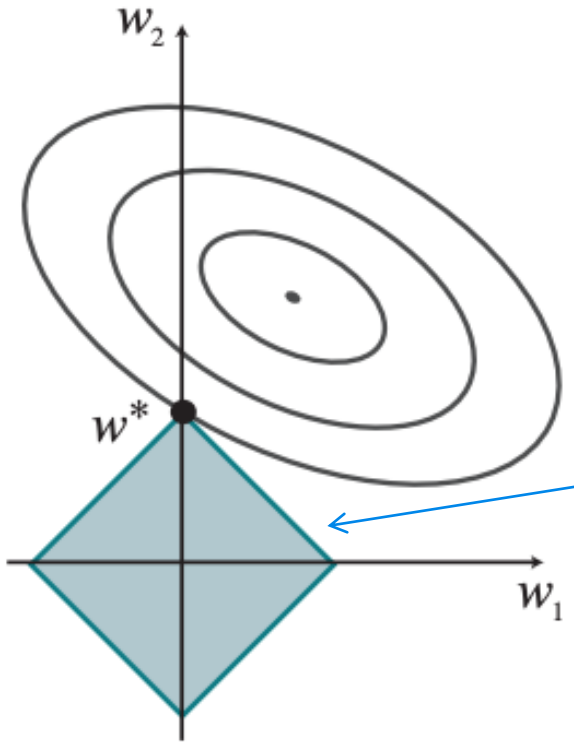
$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

L_1 regularization, q = 1
L_2 regularization, q = 2
…

Visualizing L1 Regularization

L1 regularization has a different shape:

L1 regularization creates a sparse model – many weights are zero (essentially discarded)



Set of weights w_1 and w_2 that have L_1 complexity less than c